5.2 회로 컴파일 과정 분석

개요

이 문서는 PLONK 회로의 컴파일 과정을 상세히 분석합니다. 회로 컴파일은 사용자가 정의한 회로를 실제 증명 생성과 검증에 사용할 수 있는 형태로 변환하는 핵심 과정입니다. 이 과정에서 ProverKey와 VerifierKey가 생성되며, 이들은 각각 증명 생성과 검증에 필요한 전처리된 데이터를 포함합니다.

컴파일 과정의 전체 흐름

1. 컴파일 함수 호출

```
let (pk_p, (vk, _pi_pos)) = circuit.compile::<PC>(&pp)?;
```

컴파일 과정은 `Circuit` 트레이트의 `compile` 메서드를 통해 시작됩니다:

```
fn compile<PC>(
&mut self,
u_params: &PC::UniversalParams,
) -> Result<(ProverKey<F>, (VerifierKey<F, PC>, Vec<usize>)), Error>
where
F: PrimeField,
PC: HomomorphicCommitment<F>,
{
```

```
// 1. 회로 크기 결정 및 커밋 키 생성
let circuit_size = self.padded_circuit_size();
let (ck, _) = PC::trim(u_params, circuit_size, 0, None)
.map_err(to_pc_error::<F, PC>)?;


// 2. ProverKey 생성
let mut prover = Prover::<F, P, PC>::new(b"CircuitCompilation");
self.gadget(prover.mut_cs())?;
prover.preprocess(&ck)?;


// 3. VerifierKey 생성
let mut verifier = Verifier::new(b"CircuitCompilation");
self.gadget(verifier.mut_cs())?;
verifier.preprocess(&ck)?;
// 4. 결과 반환
Ok((
prover.prover_key.expect("Missing ProverKey"),
(
verifier.verifier_key.expect("Missing VerifierKey"),
verifier.cs.intended_pi_pos,
),
))
```

```
}
```

컴파일 과정의 상세 단계

단계 1: 회로 크기 결정 및 커밋 키 생성

1.1 패딩된 회로 크기 계산

```
let circuit_size = self.padded_circuit_size();
```

- 회로 크기는 2의 거듭제곱으로 패딩됩니다 (FFT 효율성을 위해)
- `simple_circuit.rs` 예제에서는 `1 << 9 = 512` 로 설정

1.2 커밋 키 생성

```
let (ck, _) = PC::trim(u_params, circuit_size, 0, None)
```

- Universal Parameters에서 특정 회로 크기에 맞는 커밋 키를 추출
- 다항식 약정 체계(KZG10 또는 IPA)에 따라 적절한 키 생성

단계 2: ProverKey 생성 과정

2.1 Prover 인스턴스 생성

```
let mut prover = Prover::<F, P, PC>::new(b"CircuitCompilation");
```

- 트랜스크립트 초기화 문자열로 “CircuitCompilation” 사용
- 빈 StandardComposer 인스턴스 생성

2.2 회로 제약 조건 구성

```
self.gadget(prover.mut_cs())?;
```

- 사용자 정의 `gadget` 함수 호출
- StandardComposer에 모든 게이트와 제약 조건 추가
- 변수, 와이어, 셀렉터 다항식 구성

2.3 Prover 전처리

```
prover.preprocess(&ck)?;
```

전처리 과정 상세:

1. **회로 패딩**

```
self.pad(domain.size() - self.n);
```

- 회로를 2의 거듭제곱 크기로 패딩
- 빈 게이트들을 영 변수로 채움

2. **셀렉터 다항식 생성**

- 각 게이트 타입별 셀렉터 다항식 계산:
- `q_m`, `q_l`, `q_r`, `q_o`, `q_c`: 산술 게이트용
- `q_arith`, `q_range`, `q_logic`: 특수 게이트용
- `q_lookup`: 룩업 테이블용
- `q_fixed_group_add`, `q_variable_group_add`: 타원곡선 연산용

3. **순열 다항식 생성**

```
let (left_sigma_poly, right_sigma_poly, out_sigma_poly, fourth_sigma_poly) =
self.perm.compute_sigma_polynomials(self.n, &domain);
```

- 와이어 간의 순열 관계를 나타내는 시그마 다항식 계산

4. **다항식 커밋**

```
let (commitments, _) = PC::commit(commit_key, polynomials.iter(), None)?;
```

- 모든 셀렉터 및 순열 다항식에 대한 커밋 생성

5. **8n 평가값 계산**

```
let  q_m_eval_8n = Evaluations::from_vec_and_domain(
domain_8n.coset_fft(&selectors.q_m), domain_8n
);
```

- 증명 생성 시 효율성을 위해 8n 크기 도메인에서 미리 평가
- 모든 셀렉터와 순열 다항식에 대해 수행

6. **소멸 다항식 계산**

```
let  v_h_coset_8n = compute_vanishing_poly_over_coset(domain_8n, domain.size() a
```

- 몫 다항식 계산에 필요한 소멸 다항식 미리 계산

단계 3: VerifierKey 생성 과정

3.1 Verifier 인스턴스 생성

```
let mut verifier = Verifier::new(b"CircuitCompilation");
```

3.2 회로 제약 조건 구성

```
self.gadget(verifier.mut_cs())?;
```

- Prover와 동일한 회로 구성
- 동일한 제약 조건과 공개 입력 위치 확보

3.3 Verifier 전처리

```
verifier.preprocess(&ck)?;
```

Verifier 전처리는 Prover보다 간소화됨:

1. **셀렉터 다항식 생성** (Prover와 동일)
2. **순열 다항식 생성** (Prover와 동일)
3. **다항식 커밋** (Prover와 동일)
4. **8n 평가값 계산 생략** (검증 시 불필요)

ProverKey와 VerifierKey 구조 분석

ProverKey 구조

```
pub struct ProverKey<F> {
pub(crate) n: usize, // 회로 크기
pub(crate) arithmetic: arithmetic::ProverKey<F>, // 산술 게이트 키
```

```
pub(crate) range_selector: (DensePolynomial<F>, Evaluations<F>),
pub(crate) logic_selector: (DensePolynomial<F>, Evaluations<F>),
pub(crate) lookup: lookup::ProverKey<F>, // 룩업 테이블 키
pub(crate) fixed_group_add_selector: (DensePolynomial<F>, Evaluations<F>),
pub(crate) variable_group_add_selector: (DensePolynomial<F>, Evaluations<F>),
pub(crate) permutation: permutation::ProverKey<F>, // 순열 키
pub(crate) v_h_coset_8n: Evaluations<F>, // 소멸 다항식 평가값
}
```

ProverKey의 특징:

- **다항식과 평가값 쌍**: 각 셀렉터는 다항식과 8n 평가값을 모두 포함
- **8n 평가값**: 증명 생성 시 FFT 연산을 피하기 위한 최적화
- **소멸 다항식**: 몫 다항식 계산을 위한 전처리된 데이터

VerifierKey 구조

```
pub  struct VerifierKey<F, PC> {
pub(crate) n: usize, // 회로 크기
pub(crate) arithmetic: arithmetic::VerifierKey<F, PC>, // 산술 게이트 키
pub(crate) range_selector_commitment: PC::Commitment,
pub(crate) logic_selector_commitment: PC::Commitment,
pub(crate) fixed_group_add_selector_commitment: PC::Commitment,
pub(crate) variable_group_add_selector_commitment: PC::Commitment,
pub(crate) permutation: permutation::VerifierKey<PC::Commitment>,
```

```
pub(crate) lookup: lookup::VerifierKey<F, PC>,

}
```

VerifierKey의 특징:

- **커밋만 저장**: 다항식 자체가 아닌 커밋만 포함
- **크기 효율성**: ProverKey보다 훨씬 작은 크기
- **검증 최적화**: 검증에 필요한 최소한의 정보만 포함

컴파일 과정의 성능 고려사항

1. 시간 복잡도

- **FFT 연산**: O(n log n) - 각 셀렉터 다항식마다 수행
- **다항식 커밋**: O(n) - 각 다항식마다 수행
- **전체 복잡도**: O(k × n log n), k는 셀렉터 다항식 개수

2. 공간 복잡도

- **ProverKey**: O(k × n) - 모든 다항식과 평가값 저장
- **VerifierKey**: O(k) - 커밋만 저장
- **메모리 사용량**: ProverKey가 VerifierKey보다 훨씬 큼

3. 최적화 기법

8n 평가값 미리 계산

```
let  q_m_eval_8n = Evaluations::from_vec_and_domain(
```

```
domain_8n.coset_fft(&selectors.q_m), domain_8n

);
```

- 증명 생성 시 FFT 연산 생략
- 메모리 사용량 증가 vs 시간 복잡도 감소 트레이드오프

소멸 다항식 전처리

```
let  v_h_coset_8n = compute_vanishing_poly_over_coset(domain_8n, domain.size() a
```

- 몫 다항식 계산 시 반복 계산 방지

컴파일 결과 활용

1. ProverKey 사용

```
let (proof, pi) = circuit.gen_proof::<PC>(&pp, pk_p, b"Test")?;
```

- 증명 생성 시 ProverKey 활용
- 전처리된 데이터로 효율적인 증명 생성

2. VerifierKey 사용

```
verify_proof::<BlsScalar, JubJubParameters, PC>(

&pp, verifier_data.key, &proof, &verifier_data.pi, b"Test"

)?;
```

- 증명 검증 시 VerifierKey 활용
- 컴팩트한 크기로 효율적인 검증

요구사항 충족 분석

요구사항 5.2 충족도

- ✅ **회로 컴파일 과정의 각 단계 파악**:
- 회로 크기 결정
- 커밋 키 생성
- Prover/Verifier 인스턴스 생성
- 제약 조건 구성
- 전처리 수행
- ✅ **ProverKey와 VerifierKey 생성 방식 이해**:
- ProverKey: 다항식 + 평가값 + 전처리 데이터
- VerifierKey: 커밋 + 메타데이터
- 각각의 역할과 최적화 기법 분석

결론

PLONK 회로 컴파일 과정은 다음과 같은 핵심 특징을 가집니다:

1. **이중 전처리**: Prover와 Verifier 모두 전처리 수행하지만 서로 다른 최적화
2. **효율성 중심 설계**: 증명 생성/검증 시간을 단축하기 위한 전처리 투자
3. **메모리-시간 트레이드오프**: ProverKey는 크지만 빠른 증명 생성 지원
4. **표준화된 인터페이스**: Circuit 트레이트를 통한 일관된 컴파일 과정

이러한 설계를 통해 PLONK는 복잡한 회로에 대해서도 효율적인 증명 생성과 검증을 가능하게 합니다.